

PROYECTO FINAL ▪ CS3014 ▪ UTEC

Ray tracing acelerado con BVH

Escenas dinámicas y comparación de tres *backends*

BVH propio (CPU) ▪ OpenCL (GPU) ▪ Intel Embree

Kalos Lazo ▪ **Marcelo Chinchá** ▪ **Lenin Chávez**

Universidad de Ingeniería y Tecnología ▪ 2026-1

Agenda

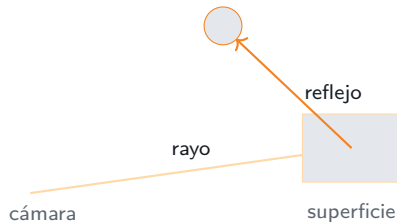
1. Introducción
2. El problema
3. Algoritmo
4. Entorno de implementación
5. Resultados
6. Demo y conclusiones

PARTE 1

Introducción

¿Qué es el ray tracing?

- ▶ Técnica de renderizado que **sigue el camino de la luz**: por cada píxel se lanza un *rayo* desde la cámara hacia la escena.
- ▶ Al chocar con una superficie se calcula sombra, y se pueden generar **rayos secundarios** (reflejo, iluminación indirecta).
- ▶ Produce sombras e iluminación globales de forma física, a cambio de un costo de cómputo alto.



Contexto: dónde se usa

- ▶ **Cine y animación:** render de producción (Pixar, ILM).
- ▶ **Videojuegos:** iluminación por trazado en tiempo real (NVIDIA RTX, consolas actuales).
- ▶ **Diseño y arquitectura:** previsualización fotorrealista.

Ese es el punto de contacto con el curso: la estructura de datos es lo que lo vuelve viable.

El cuello de botella

El costo domina por el número de *intersecciones rayo-geometría*. Su viabilidad depende de una **estructura de aceleración**.

PARTE 2

El problema

El ray tracing ingenuo es intratable

Sin aceleración, cada rayo se prueba contra **todos** los triángulos:

$$T_{\text{frame}} \approx \underbrace{P}_{\text{píxeles}} \cdot \underbrace{N}_{\text{triángulos}} \cdot (\text{rayos por píxel})$$

Orden de magnitud

1280×720 con 200 000 triángulos $\Rightarrow \sim 1,8 \times 10^{11}$ pruebas por frame. Inviabile en tiempo real con fuerza bruta $O(P \cdot N)$.

- ▶ Necesitamos **descartar grupos enteros** de triángulos con pruebas baratas.
- ▶ $\Rightarrow$ una estructura de aceleración jerárquica: el **BVH**.

Escenas estáticas, dinámicas y mixtas

Geometría estática

Edificios, terreno, objetos fijos. El árbol se **construye una vez** y se reutiliza en todos los frames $\Rightarrow$ el build se *amortiza*.

Geometría dinámica

Personajes, peatones, *mesh* deformado por animación. Cambia cada frame $\Rightarrow$ el árbol se **reconstruye cada frame** y el build *no* se amortiza.

La pregunta que guía el proyecto

Una escena real es **mixta**: parte fija + parte en movimiento. ¿Qué heurística de construcción conviene para cada una, y qué *hardware* recorre mejor el árbol?

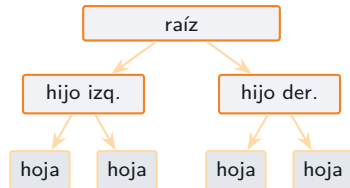
PARTE 3

Algoritmo

BVH: la idea

- ▶ Árbol binario de **cajas alineadas a los ejes** (AABB).
- ▶ La raíz envuelve toda la escena; cada nodo **reparte** sus triángulos en 2 hijos.
- ▶ Las **hojas** guardan pocos triángulos.
- ▶ El rayo prueba primero las cajas y solo desciende por las que golpea.

Costo por rayo: $O(N) \rightarrow \sim O(\log N)$.



Dos pruebas básicas: AABB y triángulo

Rayo – AABB (*slabs*). Una caja = 3 bandas (X, Y, Z). El rayo entra sii el intervalo de entrada no supera al de salida:

$$t_{\min} = \max(\text{ent.}) \leq t_{\max} = \min(\text{sal.})$$

Se precalcula $\text{inv} = 1/\mathbf{D}$ por rayo (la división es cara) y luego solo se multiplica.

Ambas son $O(1)$: el BVH sirve para *invocarlas pocas veces*, no para abaratarlas.

Rayo – triángulo (Möller–Trumbore).

Resuelve (t, u, v) de

$$\mathbf{O} + t\mathbf{D} = \mathbf{v}_0 + u\mathbf{e}_1 + v\mathbf{e}_2.$$

- ▶ $u, v \geq 0, u+v \leq 1$ es la prueba de estar dentro.
- ▶ (u, v) son los **pesos baricéntricos**: interpolan normal, color y UV.

Construcción *top-down* y las tres heurísticas

Con todos los triángulos en la raíz, mientras un nodo tenga $>$ hoja mínima:

1. eje de mayor extensión de centroides;
2. **plano de corte** $\Rightarrow$ *depende de la heurística*;
3. reparto en 2 hijos y recursión.

Comparten todo; solo cambia **dónde cortar**.

SAH

★ mejor árbol

Minimiza costo = $\text{área} \times n.^{\circ}\text{tri}$ (*binning*, 12 cubos). Build lento.

Median

Corta por la mediana de centroides. Simple; ignora el tamaño.

Morton

Ordena por **código Z** y parte el índice a la mitad. Build rapidísimo; árbol mediocre.

SAH: heurística de área de superficie

Intuición: un rayo entra en un hijo con probabilidad $\propto \text{área}(\text{hijo})/\text{área}(\text{padre})$. El costo esperado de un corte es:

$$C_{\text{split}} = 1 + \frac{A_L n_L + A_R n_R}{A_{\text{padre}}}$$

Binning: en vez de probar los $O(N)$ cortes posibles, se agrupan los centroides en **12 cubos** y se evalúan 11 cortes $\Rightarrow$ build $O(N \log N)$, cercano al óptimo.



Median split: partir por la mitad de los centroides

Intuición: dejar **la misma cantidad** de triángulos a cada lado.

1. Eje de mayor extensión de centroides.
2. **Quickselect** reordena los triángulos: la primera mitad son los de centroide menor en ese eje.
3. Corte en el **índice medio** (mitad/mitad).

Build $O(N)$ por nivel. Ignora el *tamaño* de las cajas y el espacio vacío $\Rightarrow$ hijos que se solapan $\Rightarrow$ traversal peor que SAH.



Morton / LBVH: ordenar por una curva que llena el espacio

Intuición: recorrer el espacio 3D con la **curva Z**, de modo que triángulos *vecinos* queden *contiguos* en un arreglo; luego partir por la mitad del índice.

1. Centroide $\rightarrow$ normalizar a $[0, 1]^3$; cuantizar cada eje a **10 bits**.
2. **Intercalar** los bits de x, y, z : código Morton de 30 bits ($\dots x_1 y_1 z_1 x_0 y_0 z_0$).
3. **Ordenar** una vez por ese código y cortar por el índice medio.

Build rapidísimo (domina el *sort*); los cortes por índice ignoran área y solapamiento $\Rightarrow$ árbol mediocre. Es la base de los constructores de BVH en GPU.



curva Z (orden Morton)

Recorrido con pila explícita y poda

- ▶ Recorrido **iterativo** con pila fija (sin recursión).
- ▶ **Poda 1:** caja no golpeada $\Rightarrow$ se salta el subárbol.
- ▶ **Poda 2:** caja más lejos que el mejor $t \Rightarrow$ se salta.
- ▶ En la hoja: Möller–Trumbore a sus pocos triángulos.

La misma rutina, con una pila explícita, es la que luego portamos a la GPU.

```
stack.push(raiz);
while (pila no vacia) {
    nodo = pop();
    if (!aabb_hit(nodo.box, inv, best))
        continue; // poda
    if (es_hoja) test_tris(nodo);
    else        push(hijos);
}
```


Nuestra combinación: dos BVH (estático + dinámico)

BVH estático — SAH

- ▶ Escena fija (campo, edificios).
- ▶ Se construye **una vez** con SAH (mejor árbol; el build se amortiza).

BVH dinámico — Morton

- ▶ Objetos que se mueven / *mesh* skinneado.
- ▶ Se **reconstruye cada frame**; Morton por su build barato.

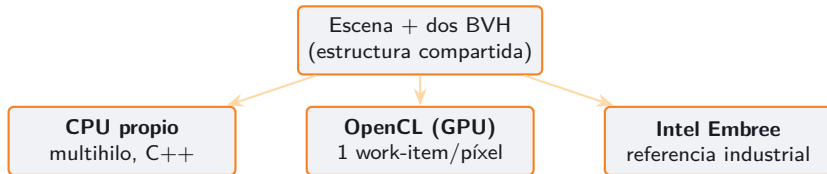
Cada rayo consulta **ambos** árboles y se queda con el golpe más cercano.

Es una TLAS de dos instancias hecha a mano: la heurística se elige según la frecuencia de cambio.

PARTE 4

Entorno de implementación

Tres *backends* sobre la misma escena



- ▶ El mismo raytracer (Möller–Trumbore, sombreado, NEE) corre en los tres.
- ▶ CPU y GPU recorren **nuestro** árbol; Embree construye el suyo.
- ▶ Se cambia de backend **en vivo** con la tecla G.

¿Qué es cada *backend*?

Backend	Quién lo escribió	Dónde corre	Qué árbol usa
CPU propio	Nosotros (C++)	CPU, 16 hilos	nuestro BVH
GPU OpenCL	Nosotros (<i>kernel</i>)	GPU, 1 hilo/píxel	nuestro (aplanado)
Embree	Intel (librería)	CPU, SIMD (AVX)	su propio BVH

- ▶ CPU y GPU son **el mismo raytracer nuestro**: cambia solo el hardware que recorre *nuestro* árbol.
- ▶ Embree es la **referencia industrial** (no es código nuestro): arma y recorre su propio árbol.
- ▶ Los tres son **ray tracing por software**: ninguno usa núcleos RT dedicados.

GPU: por qué el algoritmo cambia (*flatten*)

La GPU **no es un CPU rápido**: sin punteros de host ni recursión. Hay que **aplanar** el árbol a arrays paralelos:

- ▶ `node_bounds`: 2 float4/nodo (min, max).
- ▶ `node_links`: int4 = (izq, der, inicio, cuenta); $izq < 0 \Rightarrow$ hoja.
- ▶ `tris`: 32 float/triángulo.

El kernel recorre con una **pila explícita**; modelo **SIMT**: un *work-item* por píxel (grupos 8×8 , rayos vecinos comparten caché).

```
int stack[64]; int sp=0;
stack[sp++]=0;           // raiz
while(sp>0){
    int4 lk = node_links[stack[--sp]];
    if(lk.x<0) test_tris(lk.z,lk.w);
    else { push(lk.x); push(lk.y); }
}
```

Embree: qué hace con el BVH

- ▶ **MBVH**: nodos de **4–8 vías** (no binario)
⇒ menos niveles.
- ▶ **SIMD**: con una instrucción AVX prueba **varias cajas a la vez**.
- ▶ **Layout** compacto y nodos cuantizados
⇒ menos fallos de caché.
- ▶ Calidad de build seleccionable: HIGH (SBVH) / MEDIUM / LOW, que mapeamos a SAH / Median / Morton.

Por qué es tan rápido

- ▶ **No** por un árbol mejor: su calidad es comparable a nuestra SAH.
- ▶ Sí por la **ingeniería del kernel** (SIMD + MBVH + caché).

PARTE 5

Resultados

Setup experimental

Plataforma

- ▶ CPU: **AMD Ryzen 7 5700X3D** (8 núcleos, 16 hilos).
- ▶ GPU: **NVIDIA GTX 1650 SUPER** (OpenCL).
- ▶ Referencia: **Intel Embree 4**.

Escena: campo de esferas (densidad variable):

- ▶ Small **12 812** ■ Medium **51 212** ■ Large **204 812** tri.

Matriz completa: 3 backends $\times$ 3 heurísticas $\times$ 3 tamaños.

Metodología (reproducible: `--bench`)

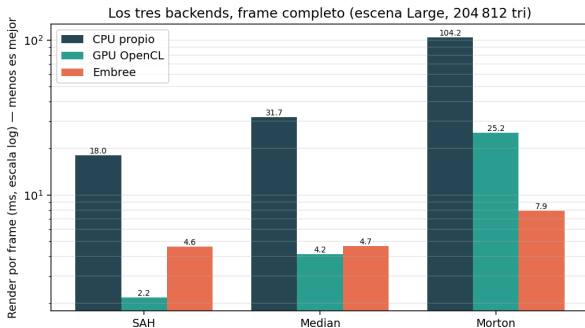
- ▶ 3 frames de *warmup* + promedio de 10.
- ▶ **render/frame**: frame completo (CPU y GPU).
- ▶ **traversal**: rayos primarios, 1 hilo (CPU y Embree).
- ▶ **nodos/rayo**: calidad del árbol, *independiente del hardware*.

Resultado principal — escena Large (204 812 tri)

Backend	Métrica	SAH	Median	Morton
CPU propio	render/frame (ms)	18,2	33,5	106,4
GPU OpenCL	render/frame (ms)	3,2	4,5	29,0
Embree	render/frame (ms)	4,6	4,7	7,9
Árbol	nodos/rayo	138	265	660

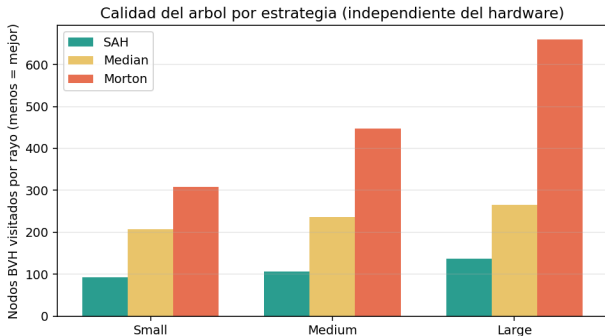
- ▶ **Heurística:** SAH visita $\sim 4,8\times$ menos nodos que Morton $\Rightarrow$ CPU $\sim 5,8\times$ más rápido.
- ▶ **GPU:** mismo árbol, $\sim 5,7\times$ sobre CPU con SAH (SIMT, un píxel por hilo).
- ▶ **Embree:** frame casi constante ante la heurística — usa su *propio* árbol robusto.

Los tres *backends*, frame completo



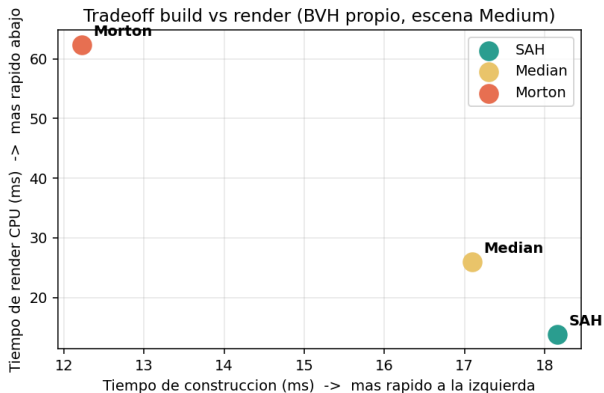
- ▶ Mismo frame completo en los tres (escala *log*).
- ▶ **CPU y GPU usan *nuestro* árbol** $\Rightarrow$ con Morton se disparan (104 y 25 ms).
- ▶ **Embree usa el *suyo*** $\Rightarrow$ apenas cambia (4,6 $\rightarrow$ 7,9 ms).
- ▶ Con buen árbol (SAH) la **GPU gana**; con árbol malo, **Embree** es el más robusto.

Calidad del árbol: SAH domina



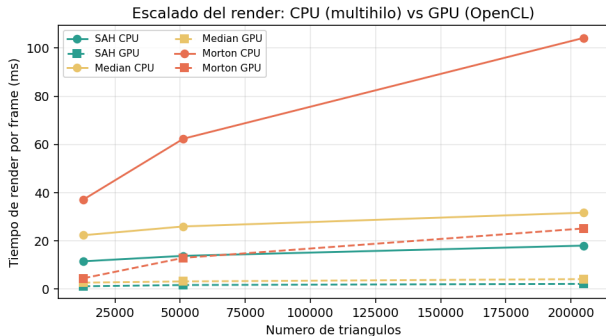
- ▶ Métrica **nodos/rayo**: independiente del hardware.
- ▶ SAH es la más baja en *todas* las densidades.
- ▶ En Large: **SAH 138 vs Morton 660**.
- ▶ La GPU hereda esta calidad: recorre el *mismo* árbol.

Compromiso *build* vs traversal



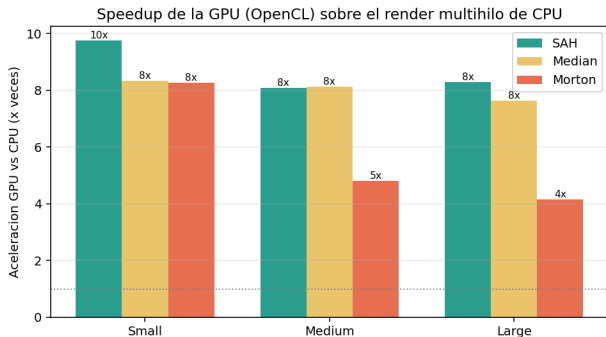
- ▶ Esquina abajo-izquierda = ideal.
- ▶ Morton: build rápido, render lento.
- ▶ SAH: build más lento, render mucho más rápido.
- ▶ Estático = se construye *1* vez, se recorre *millones* ⇒ **compensa SAH.**

Escalado: CPU vs GPU



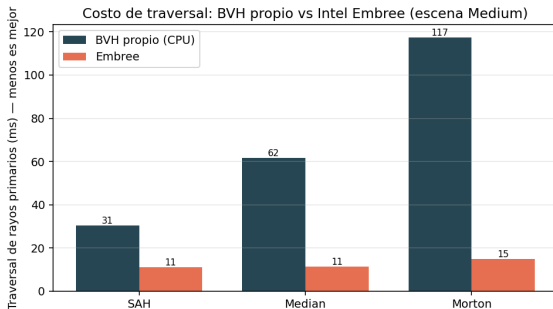
- ▶ Línea sólida: CPU; punteada: GPU (mismo árbol).
- ▶ La GPU se mantiene **plana** al crecer la escena.
- ▶ Con Morton (árbol malo), el CPU se dispara; la GPU lo absorbe mejor.

Aceleración de la GPU



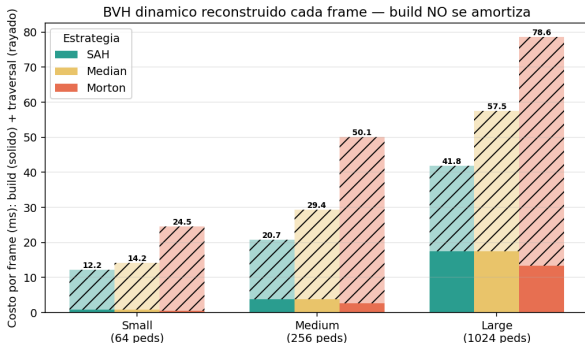
- ▶ $\text{Speedup} = \text{render CPU} / \text{render GPU (mismo árbol)}$.
- ▶ **4–10×** según heurística y tamaño.
- ▶ Es *aceleración por software*: la GTX 1650S no tiene núcleos RT dedicados.
- ▶ El paralelismo masivo por píxel gana pese al *flatten* y la pila explícita.

Embree: traversal casi constante



- ▶ Nuestro traversal escalar **sufre el árbol malo**: 31 → 116 ms de SAH a Morton.
- ▶ Embree se mantiene **~11–16 ms** pase lo que pase con el build.
- ▶ Su kernel SIMD por paquetes compensa un árbol peor.
- ▶ Lección: la brecha está en el *recorrido*, no en la estructura.

BVH dinámico: cuando el build *no* se amortiza



- ▶ Árbol **reconstruido cada frame**: costo = build + traversal.
- ▶ El build de Morton (sólido) sí es el más barato...
- ▶ ...pero aquí el build es **pequeño** frente al traversal $\Rightarrow$ **SAH gana** igual.
- ▶ No basta con volverla **dinámica**: Morton solo ganaría con *mucha* más geometría o más *rebuids/frame* (régimen *build-bound*).

PARTE 6

Demo y conclusiones

Ray tracer en tiempo real

```
./bvh_raytracer --width 640 --height 480
```

- ▶ Cambiar entre **CPU** / **GPU** / **Embree** en vivo y ver el cambio de FPS (G).
- ▶ Comparar BVH vs fuerza bruta $O(N)$ (B).
- ▶ Visualizar las cajas del árbol y una escena dinámica con personaje animado.

Conclusiones

- ▶ Ray tracer con BVH en **tres backends** (CPU, GPU OpenCL, Embree) y **benchmark reproducible** sobre la misma geometría.
- ▶ **Heurística:** la **SAH** da el mejor árbol ($\sim 4,8\times$ menos nodos/rayo que Morton)
 $\Rightarrow$ render $\sim 5,8\times$ más rápido.
- ▶ **GPU:** el *flatten* + pila explícita + SIMT logran **4–10 $\times$** sobre CPU sin hardware RT dedicado.
- ▶ **Embree:** su ventaja está en el *recorrido SIMD*, no en el árbol.
- ▶ **Dinámico:** probamos que rebuild-por-frame *no* vuelca la balanza a Morton; el traversal domina a resolución de render.

Trabajo futuro

- ▶ Recorrido **SIMD** propio (paquetes de rayos).
- ▶ **MBVH** de 4 vías, como Embree.
- ▶ Construcción **paralela** del árbol.
- ▶ **Refit** del BVH dinámico en vez de rebuild total.
- ▶ Comparar con **SBVH** (spatial splits).
- ▶ Medir el régimen *build-bound* donde Morton gana.

¡Gracias!

Preguntas y respuestas

Ray tracing acelerado con BVH ■ UTEC ■ CS3014